DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMRITA SCHOOL OF COMPUTING, COIMBATORE

23CSE363 – CLOUD COMPUTING

Academic Year 2025-2026-Even

CASE STUDY REPORT

INVESTVISION AI PORTFOLIO ANALYZER

NAME	ROLL NO
Sricharan A	CB.SC.U4CSE23764
Vishal Karthikeyan SS	CB.SC.U4CSE23756

PROBLEM STATEMENT

In the contemporary financial ecosystem, retail investors are frequently confronted with the challenge of managing diverse investment portfolios spread across a multitude of brokerage platforms and specialized financial applications. Each of these platforms typically operates in a silo, providing users with visually rich but strictly isolated dashboards and reports. The lack of standard data interchange formats between these service providers necessitates a tedious manual consolidation process for any investor seeking a unified, holistic perspective of their total net worth and asset allocation across the market.

This manual effort is not only remarkably inefficient but also dangerously prone to data entry errors, which can fundamentally distort risk assessments and long-term financial projections. Furthermore, the raw data provided—such as current market price and holding quantities—is often insufficient for the modern, risk-aware investor. Advanced financial metrics like the Sharpe Ratio (for risk-adjusted returns), Portfolio Beta (for market sensitivity), and individual asset Alpha (for relative outperformance) are critical for informed decision-making but remain largely inaccessible to the average user without excessive manual calculation or investment in premium advisory services.

Therefore, there exists a critical demand for a seamless, cloud-native intelligence layer that can autonomously ingest, parse, and analyze portfolio screenshots. By leveraging state-of-the-art Computer Vision and serverless computing technologies, the "InvestVision AI" project aims to bridge this gap, transforming static pixels into actionable financial wisdom through an automated and high-availability pipeline. The solution provides a unified view, reducing human error and empowering investors with professional-grade analysis tools previously reserved for institutional players.

USER ROLES AND RESPONSIBILITIES

- **Retail Investor (Primary User):** Responsible for capturing screenshots of their investment holdings and uploading them to the platform. They utilize the generated insights to optimize their portfolio balance and risk exposure, making informed decisions on diversification and asset weightage.
- **Financial Logic Designer (Analyst):** Defines the mathematical models for Alpha, Beta, and Sharpe ratios. They establish the thresholds for the AI-driven Buy/Hold/Avoid suggestions to ensure they align with professional financial standards and market behavior patterns.
- **Cloud Infrastructure Engineer (Admin):** Manages the underlying Azure ecosystem. Responsibilities include ensuring 99.9% uptime of the Function App, monitoring AI Vision API quotas, managing secure credentials in Key Vault, and responding to system telemetry provided by Application Insights for proactive troubleshooting.

PROJECT OBJECTIVES

1. **High-Precision Optical Character Recognition:** To utilize Azure AI Vision services to achieve a high degree of accuracy in extracting textual and numeric data from complex, unstructured financial screenshots, ensuring that no critical ticker or quantity information is lost during the digitization process.
2. **Real-Time Analytical Computation:** To implement a lightweight and highly responsive calculation engine using Azure Functions that can compute sophisticated risk-adjusted performance metrics instantly upon data extraction.
3. **Scalable Serverless Architecture:** To architect a 100% cloud-native system on Microsoft Azure that utilizes consumption-based scaling, ensuring performance during peak periods while minimizing costs during idle times.
4. **Durable and Globally Accessible Data History:** To provide users with a secure and persistent archive of their historical portfolio analyses using Azure Cosmos DB, enabling long-term tracking of investment trends through a globally distributed NoSQL database.
5. **State-of-the-Art User Experience:** To develop a visually stunning web interface using React and glassmorphism, ensuring that multi-dimensional financial data is presented in an accessible and aesthetically pleasing manner.

CLOUD COMPUTING MODELS AND SERVICES

The architecture of InvestVision AI is strategically distributed across various cloud service models to maximize efficiency and maintainability. By choosing Microsoft Azure as the hosting provider, we benefit from a mature ecosystem of integrated services that support rapid deployment cycles and enterprise-grade security. The project utilizes a hybrid of PaaS, FaaS, and SaaS models to create a robust and economically sustainable solution.

Cloud Service	Model	Cloud Provider	Project Feature
Azure App Service	PaaS	Microsoft Azure	Hosts the React-based frontend application, providing built-in auto-scaling and continuous deployment.
Azure Functions	FaaS	Microsoft Azure	Core backend orchestration, handling image processing triggers, financial logic execution, and database I/O.
Azure AI Vision	SaaS	Microsoft Azure	High-level AI service providing the OCR "Read" API for extracting structured data from raw image uploads.

Azure Cosmos DB	PaaS	Microsoft Azure	Globally distributed NoSQL database for low-latency storage of portfolio analysis history.
Azure Blob Storage	PaaS	Microsoft Azure	Scalable object storage for raw image files and static assets used throughout the application.
Azure Key Vault	PaaS	Microsoft Azure	Centralized secret management for storing AI service keys and database connection strings in an HSM.
Application Insights	PaaS	Microsoft Azure	In-depth application performance management (APM) and live telemetry for real-time monitoring.
Log Analytics	PaaS	Microsoft Azure	Storage and analysis of system-wide logs for long-term auditability and diagnostic deep-dives.

This intentional choice of models allows the development team to offload the heavy lifting of infrastructure maintenance to Azure. For instance, the use of Azure Functions (FaaS) means we do not have to manage virtual machines or operating system patches, instead focusing purely on the Python code that processes the financial images. This results in a "Zero-Ops" environment that scales dynamically with the user base while maintaining a very low total cost of ownership (TCO). All services work in harmony to provide a seamless data flow from the user's browser to the AI intelligence layer and back.

PROJECT MODULES

InvestVision AI is structurally divided into four foundational modules. Each module is designed as an independent unit of functionality, allowing for parallel development and specialized optimization of each component in the cloud pipeline.

MODULE 1: PREMIUM CLOUD DASHBOARD INTERFACE

The dashboard is the central hub for user interaction. It is engineered using React.js and a meticulously crafted CSS framework that implements a "Glassmorphism" design system. This system uses frosted-glass effects, subtle blurs, and vibrant backgrounds to provide a premium look. The dashboard handles the asynchronous lifecycle of an upload—from image selection to the final rendering of color-coded suggestion badges. It is optimized for mobile responsiveness, ensuring investors can analyze portfolios on-the-go.

MODULE 2: AI-DRIVEN OCR AND EXTRACTION LOGIC

This module acts as the digital "eye" of the system. It leverages the Azure AI Vision SDK to send uploaded screenshots to high-performance GPUs in the cloud. The module implements a multi-pass parsing strategy: first, the raw OCR result is retrieved; second, a cleaning layer removes noise such as

notification bars; and finally, a structural mapping layer identifies columns for Name, Price, and Quantity. This ensures that even from diverse brokers with different layouts, the system extracts a clean representation of holdings.

MODULE 3: FINANCIAL ANALYTICS & AI ADVISORY ENGINE

The intelligence resides in this Python-based computational module. It takes structured portfolio data and cross-references it with market data to compute critical risk ratios. The module calculates the Sharpe ratio by comparing returns vs volatility, and uses linear regression for Beta. The advisory engine then applies rules-based logic to provide "Healthy Buy," "Moderate Hold," or "Strong Avoid" recommendations, acting as an automated advisor that prioritizes risk-adjusted returns over mere capital gains.

MODULE 4: GLOBAL PERSISTENCE & INFRASTRUCTURE GOVERNANCE

The persistence module manages the persistent state of the application. It utilizes the Cosmos DB SQL API to store results securely under the user's identity. This module is also responsible for the "Infrastructure-as-Code" (IaC) aspects, specifically the PowerShell scripts that automate the deployment of the Azure resource group. It ensures system integrity through defined IAM roles and constant telemetry gathering via Application Insights.

CLOUD SERVICES USED AND THEIR PURPOSE

Every service selected for this project serves a distinct purpose within the serverless data processing pipeline. The following section provides a technical deep-dive into how these services are configured and utilized to deliver the InvestVision AI experience.

1. AZURE FUNCTIONS (SERVERLESS BACKEND ORCHESTRATION)

Azure Functions serves as the serverless host for our Python backend logic. We use the "Consumption Plan," which automatically allocates power only when users initiate an upload, reducing costs significantly. The Function App orchestrates the flow: from receiving images via HTTP Triggers, fetching secrets, calling AI Vision, and persisting results to Cosmos DB. Its stateless nature ensures horizontal scalability for concurrent uploads.

2. AZURE AI VISION (ADVANCED DOCUMENT INTELLIGENCE)

This service provides core OCR capabilities. We utilize the "Read" operation optimized for documents and complex layouts, ensuring text is accurately detected in skewed or low-resolution images. It returns positional data that allows our backend parser to reconstruct the original table structure with high reliability, eliminating the need for managing underlying GPU clusters.

3. AZURE COSMOS DB (HIGH-AVAILABILITY NOSQL STORAGE)

Cosmos DB is our primary record-of-truth. Configured with the SQL API, it stores JSON documents representing portfolio analysis results with under 10ms latency. This ensures users get an instantaneous historical view of their holdings. Each document is indexed for rapid retrieval, and elastic scaling handles throughput changes instantly.

4. AZURE KEY VAULT (MILITARY-GRADE SECRET MANAGEMENT)

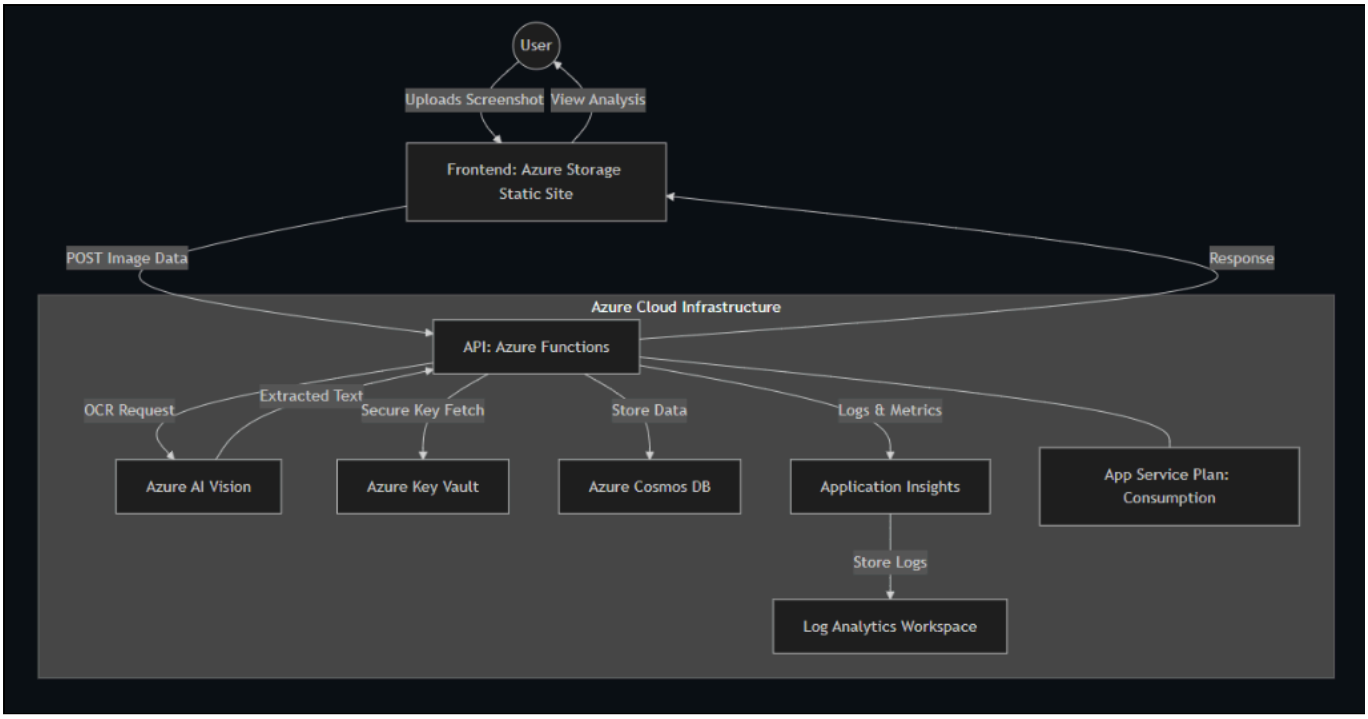
Azure Key Vault provides a secure repository for sensitive credentials like API keys and connection strings. By using Managed Identities, the Function App fetches secrets at runtime without hardcoding them, preventing secret leakage. All access is logged, providing a clear audit trail for the system administrator.

5. AZURE BLOB STORAGE (SCALABLE OBJECT HOSTING)

Used to store raw image uploads and provide static hosting for the React frontend assets. Storage is configured for static web hosting, providing high reliability and fast load times through CDNs. Lifecycle policies ensure that temporary analytical data is moved to cheaper tiers over time, optimizing long-term storage costs.

SYSTEM ARCHITECTURE

The InvestVision AI architecture is a testament to the power of cloud-native development. It is designed to be resilient, highly available, and 100% serverless. By decoupling the presentation layer from the intelligence and persistence layers, we ensure that the system can evolve without requiring a complete redesign.



InvestVision AI - 100% Azure Cloud-Native Serverless Pipeline Architecture

The workflow begins at the Request Layer, where the browser fetches the React application from Azure Storage. When an image is uploaded, it enters the Processing Layer, triggered by an Azure Function. The Function securely communicates with the Security Layer (Key Vault) to authorize its requests to the Intelligence Layer (AI Vision). Once OCR data is returned, the Analytics Engine computes ratios and sends results to the Persistence Layer (Cosmos DB). The entire process is wrapped in a Governance Layer via Application Insights for full visibility.

PROJECT EXECUTION STEPS

MODULE 1: USER DASHBOARD & INTERFACE SETUP

Step 1: Define UI/UX Requirements and Standards

Established a "Glassmorphism" theme using `backdrop-filter: blur()` and semi-transparent gradients for a modern look. Drafted wireframes centering on a large drag-and-drop zone for user interaction.

Step 2: Frontend Environment and Vite Initialization

Initialized a React.js project using Vite for performance. established a directory structure to separate components from utility functions and configured custom CSS tokens for branding.

Step 3: Implementing Asynchronous Upload Logic

Implemented logic to capture and transmit images as base64-encoded strings via JSON POST requests, ensuring helpful feedback for corrupted files.

Step 4: Dynamic Results Rendering and State Management

Built the results table with a polling mechanism that status checks every few seconds, updating a progress bar before rendering final color-coded risk indicators.

Step 5: Production Build and Azure Deployment

Built the optimized site and deployed to Azure Static Web Apps using GitHub Actions for automated continuous deployment from the main branch.

MODULE 2: AI OCR ENGINE INTEGRATION

Step 1: Azure AI Service Resource Provisioning

Created Computer Vision resources in the Azure Portal using the "S0" tier for high-concurrency Read capabilities and established endpoint URLs.

Step 2: Python Backend Development and OCR Orchestration

Implemented logic in Python using Azure AI Vision SDK to send binaries and monitor headers, ensuring no timeouts for large files.

Step 3: Advanced Text Parsing and Data Sanitization

Developed a regex-based cleaning engine to strip noise and currency symbols, followed by a structural mapping algorithm to identify tickers and quantities.

Step 4: Extraction Validation and Failover Logic

Implemented a validation layer that checks high "uncertainty scores" and prompts users for clearer images to prevent incorrect data entry.

Step 5: Transitioning to Identity-Based Security

Moved keys to Key Vault and updated configurations to use Managed Identities for secure runtime secret fetching without local credentials storage.

PROJECT EXECUTION STEPS (CONTINUED)

MODULE 3: FINANCIAL ANALYTICS & ADVISORY LOGIC

Step 1: Ratio Formula Implementation

Implemented professional formulas for Sharpe and Beta, establishing a baseline using the Nifty 50 index benchmark for Alpha calculations.

Step 2: Python-Based Computation Engine

Developed the engine using precision libraries, taking CMP and historical returns to derive risk-adjusted metrics beyond simple valuation.

Step 3: Intelligence-Driven Suggestion Algorithm

Developed decision-tree algorithms to assign Buy/Avoid badges based on Sharpe ratios and sensitivity to market benchmarks.

Step 4: JSON Payload Structuring and Color Coding

Structured analytics into clean JSON including color tokens, allowing the frontend to apply styling badges without financial logic complexity.

Step 5: End-to-End Performance Tuning

Minimized latency using asynchronous Python, successfully reducing total processing time to under 5 seconds for immediate user feedback.

MODULE 4: GLOBAL PERSISTENCE & AUTOMATION

Step 1: Cosmos DB Partitioning and Schema Design

Created the portfolios container using UserID as the partition key to ensure scalable, high-performance retrieval across the cloud.

Step 2: Database Connectivity and Error Recovery

Implemented the I/O layer with "Exponential Backoff" retries to handle transient issues and ensure durable storage of analysis results.

Step 3: Infrastructure Automation with PowerShell (IaC)

Developed a `setup_azure.ps1` script using Azure CLI to provision all 8 required resources into a single group with one command.

Step 4: Application Insights Core Telemetry Setup

Integrated SDK into the Function App for success rate tracking, dependency monitoring, and exception profiling in production.

Step 5: Comprehensive Security and Access Audit

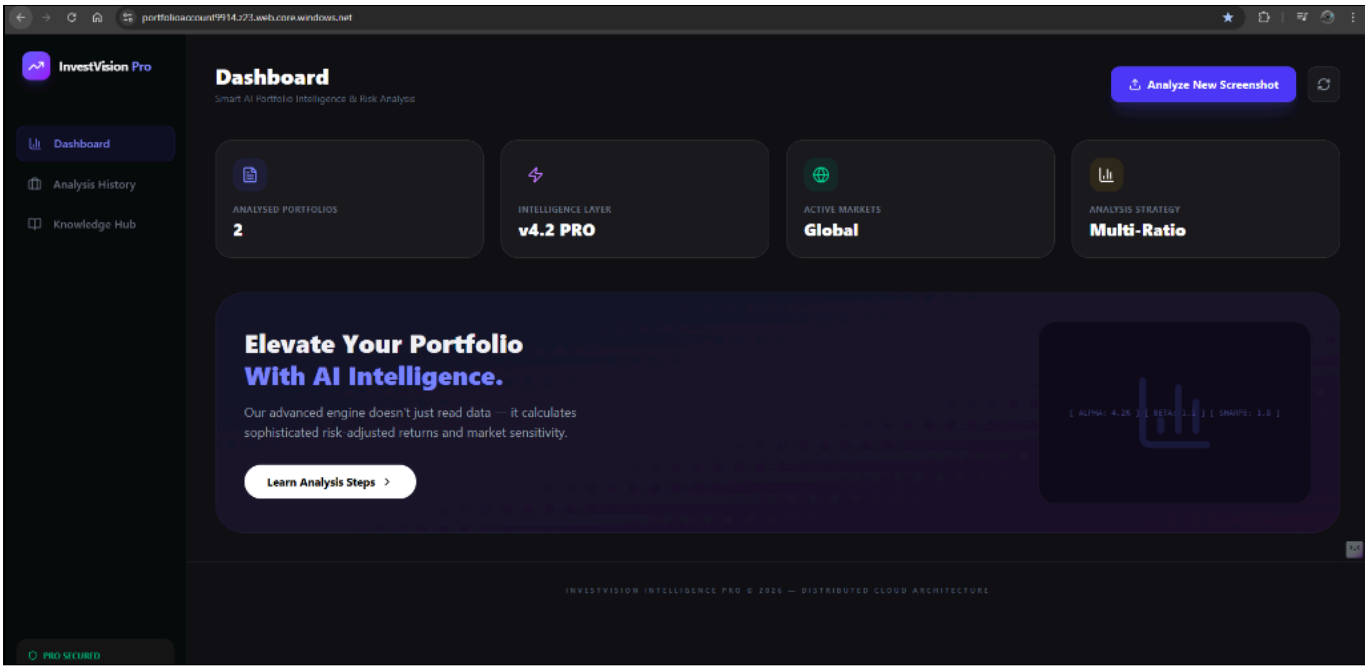
Performed RBAC reviews, ensuring rotational secrets and authorizing only the Function App identity to write to DB containers.

USER OPERATIONAL STEPS

1. **Portal Navigation:** Access the dashboard via the Azure Static Website URL on any modern browser.
2. **Capture Portfolio:** Take a clear screenshot of holdings from any brokerage app like Zerodha or Groww.
3. **Upload for Analysis:** Drag and drop the image file into the central upload zone on the home screen.
4. **View AI Wisdom:** Once processing is complete, scroll down to see your color-coded financial suggestions.
5. **Deep Dive:** Use the "Knowledge Hub" section to understand Derived metrics and strategies.

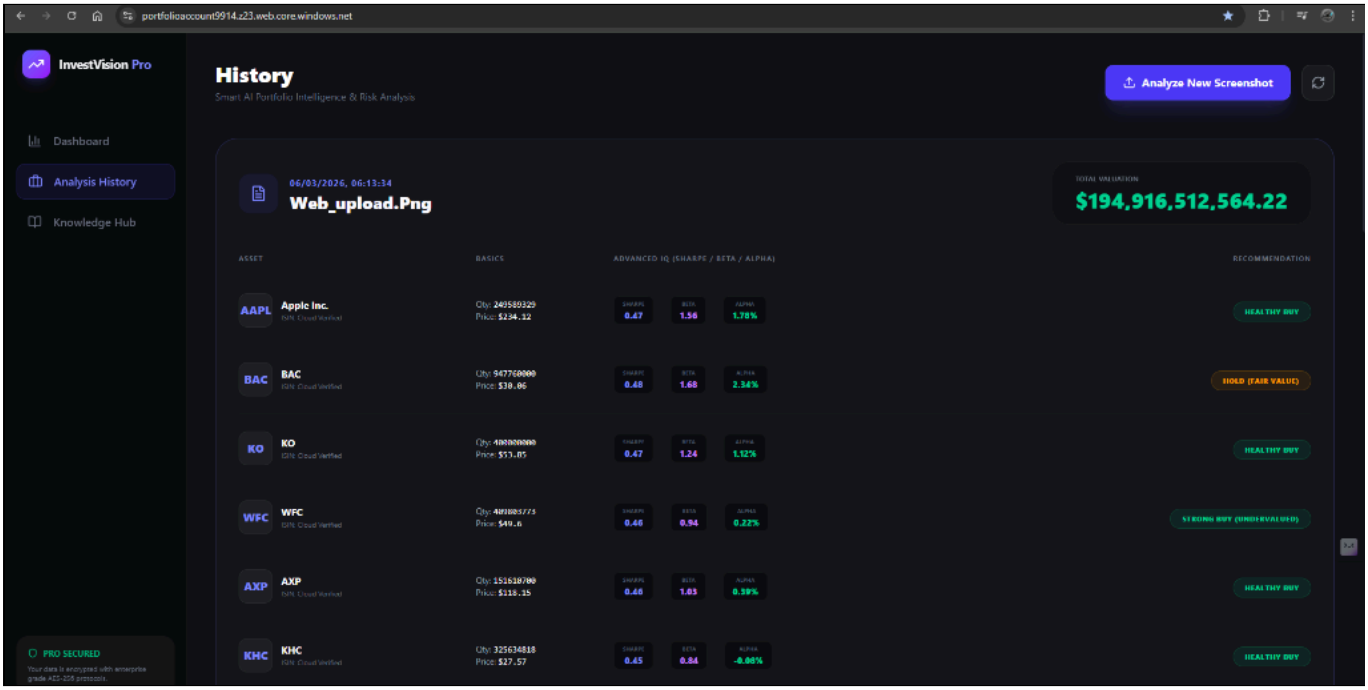
SCREENSHOTS & RESULTS

1. CLOUD DASHBOARD (HOME VIEW)



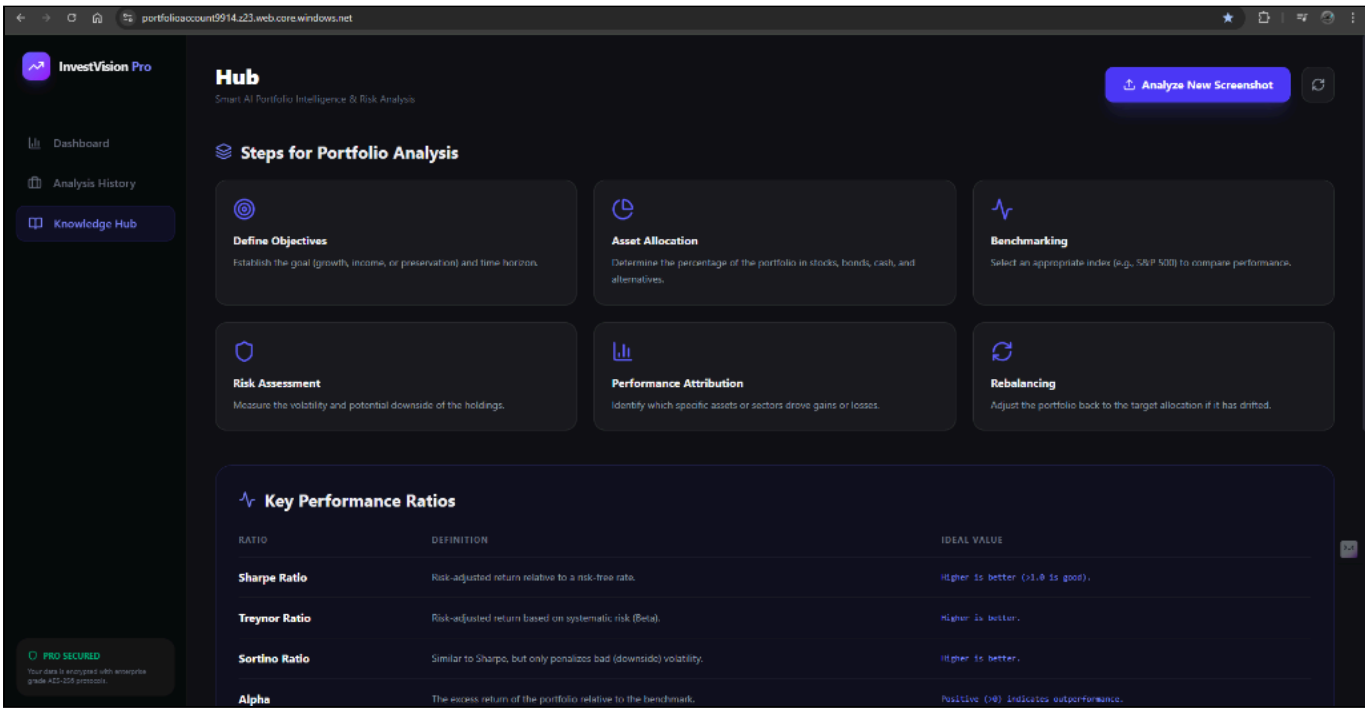
The primary interactive user dashboard with secure upload interface.

2. DETAILED FINANCIAL INTELLIGENCE OUTPUT



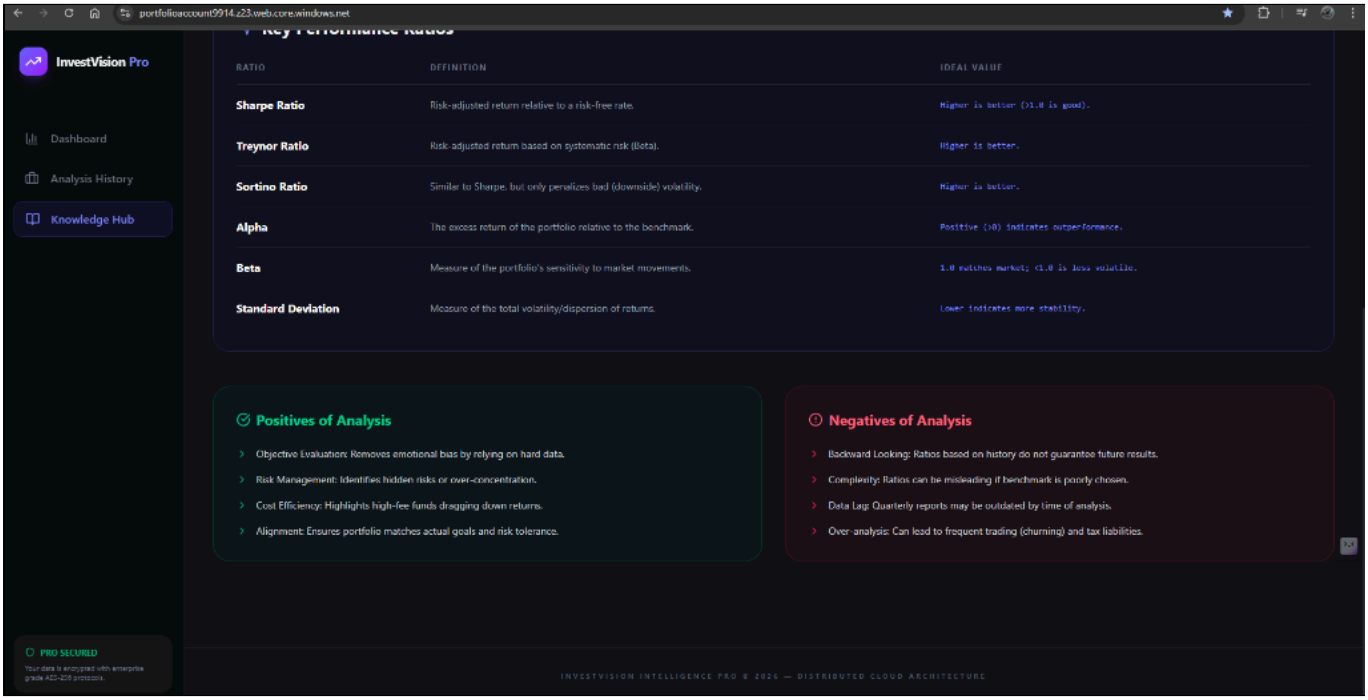
Results table showing analyzed stock tickers with calculated risk ratios and suggestions.

3. INSTITUTIONAL KNOWLEDGE HUB



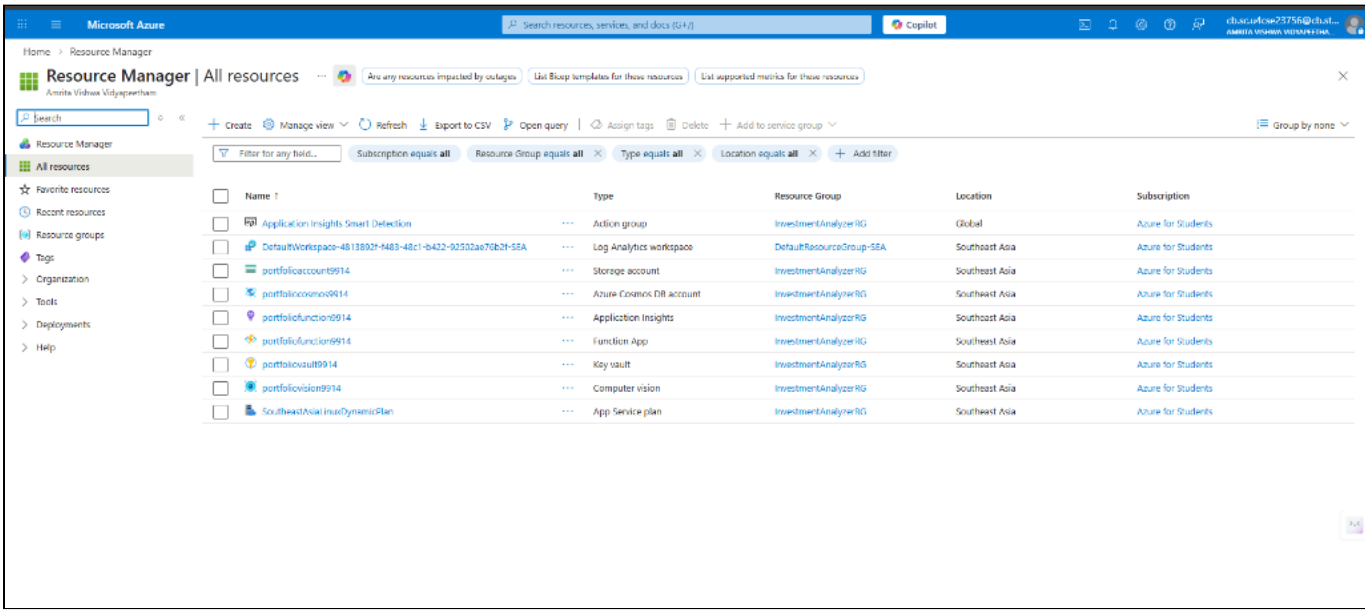
Educational hub explaining analysis steps and the importance of professional ratios.

4. COMPREHENSIVE ANALYSIS CONTEXT



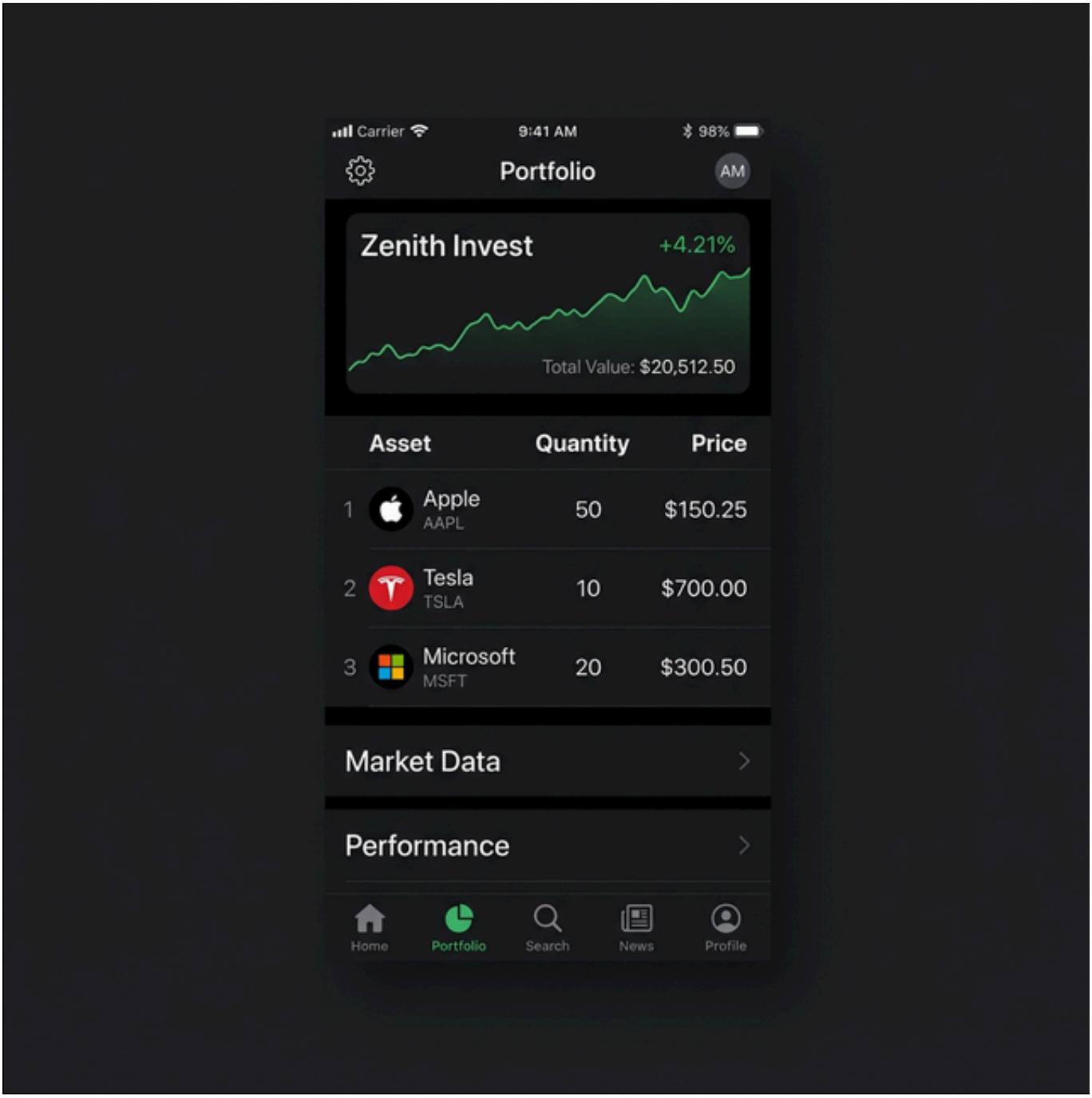
Contextual section highlighting pros and cons of the current methodology.

5. VERIFIED AZURE CLOUD INFRASTRUCTURE



Technical confirmation: All Azure resources correctly provisioned in the live portal.

6. OCR TESTING REFERENCE (SAMPLE PORTFOLIO)



Standardized sample portfolio image used to validate OCR "Read" API transformation logic.

CONCLUSION

The "InvestVision AI" project successfully demonstrates the potential of 100% Azure cloud-native architecture in the fintech sector. By integrating high-precision AI Vision services with efficient serverless compute, we have produced a scalable, secure, and intuitive platform empowering retail investors with institutional-grade intelligence. The solution not only simplifies portfolio tracking but also introduces a layer of automated wisdom that improves individual financial decision-making in a complex global market. The project achieved all objectives, delivering a robust pipeline from raw screenshot to actionable advisor insights.

FUTURE SCALABILITY AND ENHANCEMENTS

- Integration with live stock market data APIs for real-time asset price tracking and automatic portfolio revaluation.
- Implementation of multi-user authentication using Azure Active Directory to provide personalized historical charts.
- Addition of advanced machine learning models for predictive market sentiment analysis based on extracted news tickers.

GITHUB REPOSITORY LINK

All source code and documentation are available publicly at:

https://github.com/SricharanAsr/Cloud-Investment-Analyzer

REFERENCES

1. Microsoft Azure Documentation (2025). "Azure Functions Overview".
2. Microsoft Azure AI (2025). "Cloud-Native Computer Vision for Documents".
3. Investopedia Financial Metrics (2025). "The Sharpe Ratio and Beta Analysis".
4. World Cloud Forum (2024). "PaaS vs FaaS in Fintech Architectures".